EJERCICIO DE PROGRAMACIÓN
Arquitectura de Computadores

Ejercicio 2.2: Cálculo del vecino más próximo

Javier Gomez

Rodrigo Yepes

1. INTRODUCCIÓN

En este ejercicio se implementa el cálculo del vecino más próximo utilizando el dataset California Housing, aplicando técnicas de programación paralela con OpenMP.

El objetivo es analizar el rendimiento de la implementación secuencial frente a distintas versiones paralelas, evaluando el impacto del número de hilos, el tamaño del problema y las estrategias de planificación.

Además, se estudia una versión alternativa del algoritmo que reduce el consumo de memoria, así como el conteo de vecinos dentro de un radio determinado.

2. DESCRIPCIÓN DEL PROBLEMA

El problema consiste en encontrar, dada una muestra del dataset, la muestra más cercana en función de sus coordenadas geográficas (longitud y latitud).

Para ello, se calcula la distancia euclidiana entre pares de muestras y se identifica aquella con menor distancia respecto a una muestra objetivo.

Para evitar un consumo excesivo de memoria, se emplea una matriz triangular superior que almacena únicamente las distancias necesarias.

Además, se considera una versión alternativa donde únicamente se calculan las distancias respecto a la muestra objetivo, reduciendo así el coste computacional y de memoria.

3. EJECUCIÓN BASE

Comando que he ejecutado

./vecino_mas_proximo housing.csv 10 5000 first 0.1 4 64 12345

```
PC@JGM UCRT64 /c/Users/PC/OneDrive/Documentos/tercero de carrera/arquitectura de computadores/JavierGomez/Vecinomasproximo
$ ./vecino_mas_proximo housing.csv 10 5000 first 0.1 4 64 12345
========================================
CONFIGURACION DEL EXPERIMENTO
========================================
CSV: housing.csv
Total muestras en el CSV: 20640
N usado: 5000
Indice objetivo: 10
Submuestreo: first
Radio R: 0.100000
Hilos OpenMP: 4
Chunk: 64
========================================

===== 1) SOLUCION SECUENCIAL =====
Tiempo total secuencial: 0.096000 s
Muestra objetivo idx=10 -> (longitude=-122.260000, latitude=37.850000)
Vecino mas cercano (secuencial) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Vecinos dentro de R para la muestra objetivo: 513

===== 2) 3) 4) VERSIONES PARALELAS =====
---- schedule(static, 64) ----
Tiempo calculo matriz distancias: 0.020000 s
Tiempo busqueda vecino mas cercano: 0.001000 s
Vecino mas cercano (paralelo) idx=496 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Tiempo conteo vecinos dentro de R: 0.016000 s
Vecinos dentro de R para la muestra objetivo: 513

---- schedule(dynamic, 64) ----
Tiempo calculo matriz distancias: 0.020000 s
Tiempo busqueda vecino mas cercano: 0.000000 s
Vecino mas cercano (paralelo) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Tiempo conteo vecinos dentro de R: 0.014000 s
Vecinos dentro de R para la muestra objetivo: 513

===== 5) VERSION ALTERNATIVA: SOLO DISTANCIAS A LA MUESTRA OBJETIVO =====
Secuencial solo objetivo: 0.000000 s
Vecino (solo objetivo secuencial) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000

Paralelo critical - schedule(static): 0.001000 s
Vecino (critical) idx=496 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Paralelo reduction - schedule(static): 0.000000 s
Vecino (reduction) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000

Paralelo critical - schedule(dynamic): 0.000000 s
Vecino (critical) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Paralelo reduction - schedule(dynamic): 0.000000 s
Vecino (reduction) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
```

En esta ejecución se muestra el funcionamiento completo del programa utilizando un subconjunto de 5000 muestras seleccionadas de forma secuencial (first).

Se observa el tiempo de ejecución de la versión secuencial y de las versiones paralelas con diferentes estrategias de planificación (static y dynamic), así como el cálculo del vecino más cercano y el número de vecinos dentro de un radio determinado.

También se incluye la ejecución de la versión alternativa, donde únicamente se calculan las distancias respecto a la muestra objetivo.

4. EJECUCIÓN CON SUBMUESTREO ALEATORIO

Comando que he ejecutado

./vecino_mas_proximo housing.csv 10 5000 random 0.1 4 64 12345

```
PC@JGM UCRT64 /c/Users/PC/OneDrive/Documentos/tercero de carrera/arquitectura de computadores/JavierGomez/Vecinomasproximo
$ ./vecino_mas_proximo housing.csv 10 5000 random 0.1 4 64 12345
========================================
CONFIGURACION DEL EXPERIMENTO
========================================
CSV: housing.csv
Total muestras en el CSV: 20640
N usado: 5000
Indice objetivo: 10
Submuestreo: random
Radio R: 0.100000
Hilos OpenMP: 4
Chunk: 64
========================================

===== 1) SOLUCION SECUENCIAL =====
Tiempo total secuencial: 0.086000 s
Muestra objetivo idx=10 -> (longitude=-122.140000, latitude=37.710000)
Vecino mas cercano (secuencial) idx=3182 -> (longitude=-122.130000, latitude=37.710000)
Distancia minima: 0.010000000000
Vecinos dentro de R para la muestra objetivo: 72

===== 2) 3) 4) VERSIONES PARALELAS =====
---- schedule(static, 64) ----
Tiempo calculo matriz distancias: 0.017000 s
Tiempo busqueda vecino mas cercano: 0.001000 s
Vecino mas cercano (paralelo) idx=3182 -> (longitude=-122.130000, latitude=37.710000)
Distancia minima: 0.010000000000
Tiempo conteo vecinos dentro de R: 0.014000 s
Vecinos dentro de R para la muestra objetivo: 72

---- schedule(dynamic, 64) ----
Tiempo calculo matriz distancias: 0.014000 s
Tiempo busqueda vecino mas cercano: 0.000000 s
Vecino mas cercano (paralelo) idx=3182 -> (longitude=-122.130000, latitude=37.710000)
Distancia minima: 0.010000000000
Tiempo conteo vecinos dentro de R: 0.013000 s
Vecinos dentro de R para la muestra objetivo: 72

===== 5) VERSION ALTERNATIVA: SOLO DISTANCIAS A LA MUESTRA OBJETIVO =====
Secuencial solo objetivo: 0.000000 s
Vecino (solo objetivo secuencial) idx=3182 -> (longitude=-122.130000, latitude=37.710000)
Distancia minima: 0.010000000000

Paralelo critical - schedule(static): 0.000000 s
Vecino (critical) idx=3182 -> (longitude=-122.130000, latitude=37.710000)
Distancia minima: 0.010000000000
Paralelo reduction - schedule(static): 0.000000 s
Vecino (reduction) idx=3182 -> (longitude=-122.130000, latitude=37.710000)
Distancia minima: 0.010000000000

Paralelo critical - schedule(dynamic): 0.001000 s
Vecino (critical) idx=3182 -> (longitude=-122.130000, latitude=37.710000)
Distancia minima: 0.010000000000
Paralelo reduction - schedule(dynamic): 0.000000 s
Vecino (reduction) idx=3182 -> (longitude=-122.130000, latitude=37.710000)
Distancia minima: 0.010000000000


PC@JGM UCRT64 /c/Users/PC/OneDrive/Documentos/tercero de carrera/arquitectura de computadores/JavierGomez/Vecinomasproximo
```

En esta ejecución se utiliza un submuestreo aleatorio (random) en lugar de tomar las primeras N muestras.

Esto permite analizar cómo influye la selección de datos en los resultados, tanto en el vecino más cercano encontrado como en los tiempos de ejecución.

5. ESCALABILIDAD CON NÚMERO DE HILOS

Comandos que he ejecutado

./vecino_mas_proximo housing.csv 10 5000 first 0.1 1 64 12345
./vecino_mas_proximo housing.csv 10 5000 first 0.1 2 64 12345
./vecino_mas_proximo housing.csv 10 5000 first 0.1 4 64 12345

```
PC@JGM UCRT64 /c/Users/PC/OneDrive/Documentos/tercero de carrera/arquitectura de computadores/JavierGomez/Vecinomasproximo
$ ./vecino_mas_proximo housing.csv 10 5000 first 0.1 1 64 12345
./vecino_mas_proximo housing.csv 10 5000 first 0.1 2 64 12345
./vecino_mas_proximo housing.csv 10 5000 first 0.1 4 64 12345
========================================
CONFIGURACION DEL EXPERIMENTO
========================================
CSV: housing.csv
Total muestras en el CSV: 20640
N usado: 5000
Indice objetivo: 10
Submuestreo: first
Radio R: 0.100000
Hilos OpenMP: 1
Chunk: 64
========================================

===== 1) SOLUCION SECUENCIAL =====
Tiempo total secuencial: 0.090000 s
Muestra objetivo idx=10 -> (longitude=-122.260000, latitude=37.850000)
Vecino mas cercano (secuencial) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Vecinos dentro de R para la muestra objetivo: 513

===== 2) 3) 4) VERSIONES PARALELAS =====
---- schedule(static, 64) ----
Tiempo calculo matriz distancias: 0.039000 s
Tiempo busqueda vecino mas cercano: 0.000000 s
Vecino mas cercano (paralelo) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Tiempo conteo vecinos dentro de R: 0.032000 s
Vecinos dentro de R para la muestra objetivo: 513

---- schedule(dynamic, 64) ----
Tiempo calculo matriz distancias: 0.035000 s
Tiempo busqueda vecino mas cercano: 0.000000 s
Vecino mas cercano (paralelo) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Tiempo conteo vecinos dentro de R: 0.035000 s
Vecinos dentro de R para la muestra objetivo: 513

===== 5) VERSION ALTERNATIVA: SOLO DISTANCIAS A LA MUESTRA OBJETIVO =====
Secuencial solo objetivo: 0.000000 s
Vecino (solo objetivo secuencial) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000

Paralelo critical - schedule(static): 0.000000 s
Vecino (critical) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Paralelo reduction - schedule(static): 0.000000 s
Vecino (reduction) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000

Paralelo critical - schedule(dynamic): 0.000000 s
Vecino (critical) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Paralelo reduction - schedule(dynamic): 0.000000 s
Vecino (reduction) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
```

```
========================================
CONFIGURACION DEL EXPERIMENTO
========================================
CSV: housing.csv
Total muestras en el CSV: 20640
N usado: 5000
Indice objetivo: 10
Submuestreo: first
Radio R: 0.100000
Hilos OpenMP: 2
Chunk: 64
========================================

===== 1) SOLUCION SECUENCIAL =====
Tiempo total secuencial: 0.087000 s
Muestra objetivo idx=10 -> (longitude=-122.260000, latitude=37.850000)
Vecino mas cercano (secuencial) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Vecinos dentro de R para la muestra objetivo: 513

===== 2) 3) 4) VERSIONES PARALELAS =====
---- schedule(static, 64) ----
Tiempo calculo matriz distancias: 0.020000 s
Tiempo busqueda vecino mas cercano: 0.000000 s
Vecino mas cercano (paralelo) idx=496 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Tiempo conteo vecinos dentro de R: 0.018000 s
Vecinos dentro de R para la muestra objetivo: 513

---- schedule(dynamic, 64) ----
Tiempo calculo matriz distancias: 0.018000 s
Tiempo busqueda vecino mas cercano: 0.000000 s
Vecino mas cercano (paralelo) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Tiempo conteo vecinos dentro de R: 0.020000 s
Vecinos dentro de R para la muestra objetivo: 513

===== 5) VERSION ALTERNATIVA: SOLO DISTANCIAS A LA MUESTRA OBJETIVO =====
Secuencial solo objetivo: 0.000000 s
Vecino (solo objetivo secuencial) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000

Paralelo critical - schedule(static): 0.000000 s
Vecino (critical) idx=496 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Paralelo reduction - schedule(static): 0.000000 s
Vecino (reduction) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000

Paralelo critical - schedule(dynamic): 0.000000 s
Vecino (critical) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Paralelo reduction - schedule(dynamic): 0.001000 s
Vecino (reduction) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
```

```
========================================
CONFIGURACION DEL EXPERIMENTO
========================================
CSV: housing.csv
Total muestras en el CSV: 20640
N usado: 5000
Indice objetivo: 10
Submuestreo: first
Radio R: 0.100000
Hilos OpenMP: 4
Chunk: 64
========================================

===== 1) SOLUCION SECUENCIAL =====
Tiempo total secuencial: 0.081000 s
Muestra objetivo idx=10 -> (longitude=-122.260000, latitude=37.850000)
Vecino mas cercano (secuencial) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Vecinos dentro de R para la muestra objetivo: 513

===== 2) 3) 4) VERSIONES PARALELAS =====
---- schedule(static, 64) ----
Tiempo calculo matriz distancias: 0.018000 s
Tiempo busqueda vecino mas cercano: 0.000000 s
Vecino mas cercano (paralelo) idx=496 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Tiempo conteo vecinos dentro de R: 0.017000 s
Vecinos dentro de R para la muestra objetivo: 513

---- schedule(dynamic, 64) ----
Tiempo calculo matriz distancias: 0.015000 s
Tiempo busqueda vecino mas cercano: 0.000000 s
Vecino mas cercano (paralelo) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Tiempo conteo vecinos dentro de R: 0.013000 s
Vecinos dentro de R para la muestra objetivo: 513

===== 5) VERSION ALTERNATIVA: SOLO DISTANCIAS A LA MUESTRA OBJETIVO =====
Secuencial solo objetivo: 0.000000 s
Vecino (solo objetivo secuencial) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000

Paralelo critical - schedule(static): 0.000000 s
Vecino (critical) idx=496 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Paralelo reduction - schedule(static): 0.000000 s
Vecino (reduction) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000

Paralelo critical - schedule(dynamic): 0.000000 s
Vecino (critical) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Paralelo reduction - schedule(dynamic): 0.000000 s
Vecino (reduction) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000


PC@JGM UCRT64 /c/Users/PC/OneDrive/Documentos/tercero de carrera/arquitectura de computadores/JavierGomez/Vecinomasproximo
$
```

Se han realizado pruebas variando el número de hilos de ejecución para analizar la escalabilidad del programa.

Se observa que al aumentar el número de hilos, el tiempo de ejecución de las versiones paralelas disminuye, lo que demuestra la existencia de paralelismo de datos en el problema.

6. VARIACIÓN DEL TAMAÑO DEL PROBLEMA (N)

Comandos que he ejecutado

./vecino_mas_proximo housing.csv 10 2000 first 0.1 4 64 12345
./vecino_mas_proximo housing.csv 10 10000 first 0.1 4 64 12345

```
/c/Users/PC/OneDrive/Documentos/tercero de carrera/arquitectura de computadores/JavierGomez/Vecinomasproximo

PC@JGM UCRT64 /c/Users/PC/OneDrive/Documentos/tercero de carrera/arquitectura de computadores/JavierGomez/Vecinomasproximo
$ ./vecino_mas_proximo housing.csv 10 2000 first 0.1 4 64 12345
./vecino_mas_proximo housing.csv 10 10000 first 0.1 4 64 12345
========================================
CONFIGURACION DEL EXPERIMENTO
========================================
CSV: housing.csv
Total muestras en el CSV: 20640
N usado: 2000
Indice objetivo: 10
Submuestreo: first
Radio R: 0.100000
Hilos OpenMP: 4
Chunk: 64
========================================

===== 1) SOLUCION SECUENCIAL =====
Tiempo total secuencial: 0.014000 s
Muestra objetivo idx=10 -> (longitude=-122.260000, latitude=37.850000)
Vecino mas cercano (secuencial) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Vecinos dentro de R para la muestra objetivo: 513

===== 2) 3) 4) VERSIONES PARALELAS =====
---- schedule(static, 64) ----
Tiempo calculo matriz distancias: 0.005000 s
Tiempo busqueda vecino mas cercano: 0.000000 s
Vecino mas cercano (paralelo) idx=496 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Tiempo conteo vecinos dentro de R: 0.003000 s
Vecinos dentro de R para la muestra objetivo: 513

---- schedule(dynamic, 64) ----
Tiempo calculo matriz distancias: 0.002000 s
Tiempo busqueda vecino mas cercano: 0.000000 s
Vecino mas cercano (paralelo) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Tiempo conteo vecinos dentro de R: 0.003000 s
Vecinos dentro de R para la muestra objetivo: 513

===== 5) VERSION ALTERNATIVA: SOLO DISTANCIAS A LA MUESTRA OBJETIVO =====
Secuencial solo objetivo: 0.000000 s
Vecino (solo objetivo secuencial) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000

Paralelo critical - schedule(static): 0.000000 s
Vecino (critical) idx=496 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Paralelo reduction - schedule(static): 0.000000 s
Vecino (reduction) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000

Paralelo critical - schedule(dynamic): 0.001000 s
Vecino (critical) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Paralelo reduction - schedule(dynamic): 0.000000 s
Vecino (reduction) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
```

```
========================================
CONFIGURACION DEL EXPERIMENTO
========================================
CSV: housing.csv
Total muestras en el CSV: 20640
N usado: 10000
Indice objetivo: 10
Submuestreo: first
Radio R: 0.100000
Hilos OpenMP: 4
Chunk: 64
========================================

===== 1) SOLUCION SECUENCIAL =====
Tiempo total secuencial: 0.347000 s
Muestra objetivo idx=10 -> (longitude=-122.260000, latitude=37.850000)
Vecino mas cercano (secuencial) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Vecinos dentro de R para la muestra objetivo: 513

===== 2) 3) 4) VERSIONES PARALELAS =====
---- schedule(static, 64) ----
Tiempo calculo matriz distancias: 0.065000 s
Tiempo busqueda vecino mas cercano: 0.001000 s
Vecino mas cercano (paralelo) idx=496 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Tiempo conteo vecinos dentro de R: 0.058000 s
Vecinos dentro de R para la muestra objetivo: 513

---- schedule(dynamic, 64) ----
Tiempo calculo matriz distancias: 0.055000 s
Tiempo busqueda vecino mas cercano: 0.001000 s
Vecino mas cercano (paralelo) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Tiempo conteo vecinos dentro de R: 0.045000 s
Vecinos dentro de R para la muestra objetivo: 513

===== 5) VERSION ALTERNATIVA: SOLO DISTANCIAS A LA MUESTRA OBJETIVO =====
Secuencial solo objetivo: 0.000000 s
Vecino (solo objetivo secuencial) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000

Paralelo critical - schedule(static): 0.000000 s
Vecino (critical) idx=496 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Paralelo reduction - schedule(static): 0.001000 s
Vecino (reduction) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000

Paralelo critical - schedule(dynamic): 0.001000 s
Vecino (critical) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000
Paralelo reduction - schedule(dynamic): 0.000000 s
Vecino (reduction) idx=11 -> (longitude=-122.260000, latitude=37.850000)
Distancia minima: 0.000000000000


PC@JGM UCRT64 /c/Users/PC/OneDrive/Documentos/tercero de carrera/arquitectura de computadores/JavierGomez/Vecinomasproximo
```

Se han realizado pruebas utilizando diferentes tamaños del problema (N), con el objetivo de analizar cómo afecta el volumen de datos al tiempo de ejecución.

Se observa que al aumentar el número de muestras, el tiempo de ejecución crece, lo que permite evaluar la complejidad del algoritmo.

7. Resultados experimentales

7.1 Escalabilidad en función del número de hilos

Hilos	Tiempo Secuencial (s)	Tiempo Paralelo (s)	Speedup
1	0.090	0.039	2.30
2	0.087	0.020	4.35
4	0.081	0.018	4.50

Se ha analizado el comportamiento del algoritmo al variar el número de hilos de ejecución.

Se observa que el tiempo de ejecución disminuye al aumentar el número de hilos, lo que demuestra que el problema presenta paralelismo de datos.

El speedup obtenido no es perfectamente lineal, debido al overhead de creación de hilos y a la sincronización entre ellos, pero muestra una mejora significativa del rendimiento.

7.2 Influencia del tamaño del problema

N	Tiempo Secuencial (s)	Tiempo Paralelo (s)
2000	0.014	0.005
5000	0.081	0.018
10000	0.347	0.065

Se observa que al aumentar el número de muestras, el tiempo de ejecución crece de forma significativa, lo que confirma la complejidad del algoritmo.

La versión paralela reduce el tiempo de ejecución en todos los casos, aunque el coste computacional sigue aumentando con el tamaño del problema.

7.3 Comparación de estrategias de planificación

N	Static (s)	Dynamic (s)
5000	0.020	0.020
10000	0.065	0.055

Se han comparado las estrategias de planificación static y dynamic.

Se observa que dynamic puede ofrecer mejores resultados en algunos casos, ya que distribuye mejor la carga de trabajo entre los hilos, evitando posibles desequilibrios.

7.4 Comparación del tipo de submuestreo

Método	Tiempo (s)	Vecinos dentro de R
First	0.081	513
Random	0.086	72

Se ha comparado el uso de submuestreo secuencial (first) frente a submuestreo aleatorio (random).

Se observa que el número de vecinos dentro del radio cambia significativamente, lo que indica que la distribución de los datos influye directamente en los resultados del algoritmo.

8. Análisis de resultados

El problema presenta paralelismo de datos, ya que el cálculo de distancias entre pares de muestras es completamente independiente. Esto permite paralelizar el algoritmo utilizando OpenMP mediante bucles paralelos.

La mejora de rendimiento observada al aumentar el número de hilos confirma la existencia de escalabilidad, aunque no es lineal debido al overhead de gestión de hilos y a la sincronización necesaria en algunas partes del código.

La comparación entre las estrategias static y dynamic muestra que dynamic puede mejorar el equilibrio de carga en determinadas situaciones, especialmente cuando la carga de trabajo no está uniformemente distribuida.

En cuanto a la memoria, el uso de una matriz triangular superior permite reducir significativamente el consumo, evitando almacenar valores redundantes.

La versión alternativa del algoritmo, en la que únicamente se calculan las distancias respecto a la muestra objetivo, reduce aún más el uso de memoria, aunque limita la reutilización de datos.

Finalmente, se observa la existencia de distancias iguales a cero en algunos casos, lo que indica la presencia de muestras con coordenadas idénticas en el dataset.

9. Conclusiones

La implementación del cálculo del vecino más próximo ha permitido comprobar la eficacia de la paralelización mediante OpenMP en problemas con alto paralelismo de datos.

Se ha observado una mejora significativa del rendimiento al utilizar múltiples hilos, así como la influencia de la estrategia de planificación y del tamaño del problema en el tiempo de ejecución.

Además, se ha comprobado que la optimización del uso de memoria mediante estructuras como la matriz triangular o la versión alternativa contribuye a mejorar la eficiencia del algoritmo.